

Machine Test: AI-Based Product Search + Authentication

Overview

Build a backend system where users can search products using **natural language queries**.

Objective

Evaluate ability to:

- Build REST APIs (Node.js)
 - Work with MySQL using ORM
 - Implement JWT authentication
 - Apply NLP/AI parsing
 - Write clean, scalable code
-

Time Limit

6–7 Hours

Tech Stack

Mandatory:

- Node.js (Express.js)
- MySQL
- REST APIs
- JWT Authentication
- **ORM (Required):**
 - Sequelize (preferred)
 - OR Prisma / TypeORM

AI/NLP:

- OpenAI / Gemini / Groq / OpenRouter
- OR rule-based parser

Functional Requirements

1. Authentication System

- POST /api/auth/register
- POST /api/auth/login

Requirements:

- bcrypt password hashing
 - JWT token
 - Protected routes
-

2. Search API (Core)

GET /api/search?q=your_query

Example:

“cheap laptop under 50000”

Expected Parsing:

- category → laptop
- max_price → 50000

Output:

Filtered products using ORM queries

3. Database (Using ORM)

Tables:

- users (id, name, email, password, role)
- categories (id, name)
- products (id, name, category_id, price, color, description, tags)

Requirements:

- Use ORM models (no raw SQL in controllers)

- Define relationships (associations)
 - Seed at least 10 products
-

4. Architecture (MVC)

- Controllers → handle request/response
 - Models → ORM models
 - Services → NLP + business logic
 - Middleware → auth & roles
-

5. Flow

1. Receive query
 2. Parse (AI or logic)
 3. Convert to filters
 4. Build ORM query
 5. Fetch data
 6. Return JSON
-

6. APIs

Required:

- POST /api/auth/register
- POST /api/auth/login
- GET /api/search?q=

Optional:

- Product CRUD APIs
-

7. Role-Based Access (Bonus)

- Add role in user
- Middleware for role validation

- Restrict product APIs

Middleware:

authMiddleware → verify JWT

roleMiddleware → restrict access

8. Dashboard APIs (NEW)

Super Admin

GET /api/dashboard/overview

total users

total products

total categories

Admin

POST /api/products

PUT /api/products/:id

DELETE /api/products/:id

GET /api/products

Manager

GET /api/dashboard/products

GET /api/search

Employee

GET /api/search

GET /api/products

Client/User

GET /api/search

GET /api/products

6. Flow

Login → JWT

Middleware validates user + role

Allow/deny API access

Search uses NLP → filters → ORM

9. Rules

-  Use ORM (Sequelize/Prisma)
 -  No hardcoded filters
 -  No raw SQL in controllers
 -  Proper error handling
 -  Clean structure
-

10. Edge Cases

- Empty query
 - Invalid query
 - No results
 - Multiple filters
-

11. Bonus

- Pagination
 - Sorting
 - Caching (Redis)
 - Full-text search
-

12. Submission

- Source code /GitHub/Zip
- DB migration/seed
- README
- Postman (optional)

